
Oliver Krauss

Document Modelling

MDB3 | MTD.ba VZ WS26

Hagenberg 01./02.10.2026

HAGENBERG | LINZ | STEYR | WELS



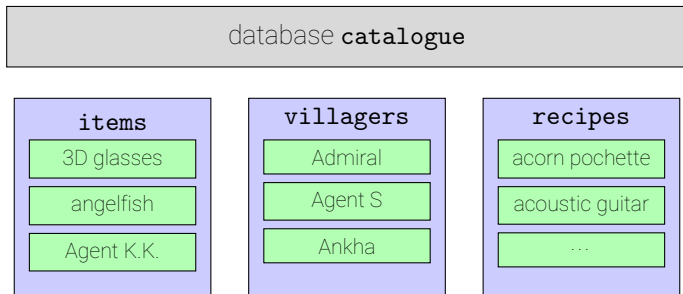
UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

The Words, Side by Side

SQL	MongoDB
database	database
table	collection
row	document
column	field
primary key	_id
index	index
JOIN	embedding, or \$lookup
GROUP BY	aggregation pipeline
schema	the documents (validation opt-in)

- A **collection** groups documents that belong together; nothing forces them to share fields
- A **document** is one JSON-like object, stored as BSON (binary JSON with more types)
- Every document has an **_id**; MongoDB makes one if you do not
- The mapping is a crutch: use it for the first week, then think in documents

Database, Collections, Documents



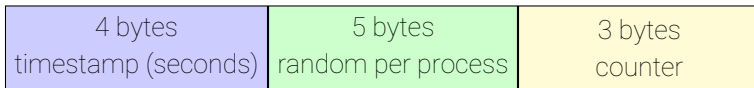
- Collections and databases appear on the first insert; there is no **CREATE TABLE** to write first
- One database per application, one collection per kind of thing: the same instinct as tables

What a Document Can Hold

```
{ "_id": ObjectId("66f1a2b3c4d5e6f708091a0b"),    // ObjectId, made by MongoDB
  "name": "Admiral",                             // string
  "species": "Bird", "personality": "Cranky",
  "birthday": { "month": 1, "day": 27 },          // embedded document
  "lastVisit": ISODate("2026-09-23T10:15:00Z"),  // date, a real one
  "styles": ["Cool"],                             // array
  "home": { "wallpaper": "dirt-clod wall",        // another embedded document
            "flooring": "tatami" },
  "furniture": [717, 1849, 7047],                 // array of ints (references)
  "popularity": 4.7, "active": true }             // double, boolean
```

- BSON adds what JSON lacks: **ObjectId**, **Date**, 32- and 64-bit integers, **Decimal128**, binary
- **Arrays and embedded documents nest without limit**: the shape of the data is the schema
- A field that does not apply is simply absent; there is no **NULL** column to fill

The `_id` and the ObjectId



- 12 bytes, unique without asking a central server: every node, every process can mint one
- Roughly sorted by creation time, so `sort({_id: 1})` is insertion order
- Any value can be `_id` as long as it is unique in the collection: the catalogue's **Unique Entry ID** would do
- `_id` is immutable and always indexed, exactly like a primary key

One Collection, Many Shapes

```
{ "name": "3D glasses", "type": "accessory", "sell": 122, "variations": ["White", "Black"] }
{ "name": "angelfish", "type": "fish", "sell": 3000, "where": "River" }
{ "name": "Agent K.K.", "type": "music", "sell": 800, "source": "K.K. concert" }
```

- **Polymorphic collection**: documents share some fields (**name**, **type**, **sell**) and differ in the rest
- A **type field** names the shape; **{type: "fish"}** selects one shape
- The shared fields are what you query and index across shapes; keep them consistent on purpose

Where this bites

Nothing stops a fourth document with **"Type"** or **"sell": "800"**. Consistency is your job, or the validator's (*Indexes and Schema Validation*).

Schema Evolution: Version by Version

v1: what the CSV had

```
{ "name": "Admiral",  
  "species": "Bird",  
  "wall": "clod",  
  "floor": "tatami" }
```

v2: the home grouped

```
{ "name": "Admiral",  
  "species": "Bird",  
  "home": {  
    "wall": "clod",  
    "floor": "tatami"  
  },  
  "schemaVersion": 2 }
```

v3: a feature added

```
{ "name": "Admiral",  
  "species": "Bird",  
  "home": { ... },  
  "visits": [  
    { "on": ISODate  
      (...),  
      "by": "player1"  
    } ],  
  "schemaVersion": 3 }
```

- Old and new documents **coexist**; nothing is locked, nothing is down
- The application reads **schemaVersion**; documents are migrated when touched or in batches
- Adding, renaming, nesting, splitting: updates on documents, not DDL on a table

Two Ways to Connect Data

Embed

Put the related data **inside** the document: the home inside the villager, the variations inside the item.

- one read, no join, atomic update
- duplicated if shared, grows with the document

The relational reflex

In SQL you normalise first and join always. In MongoDB you **embed first** and reference when embedding stops working. The rest of this block is about when that is.

Reference

Store the other document's **_id**: the furniture ids inside the villager, the items in their own collection.

- stored once, independent life
- a second query (or **\$lookup**) to read it

Embedding

```
{ "name": "acoustic guitar", "type": "houseware",  
  "sell": 3210, "diy": true, "kitCost": 5,  
  "variations": [  
    { "name": "Natural", "colors": ["Beige", "  
      Brown"] },  
    { "name": "Cherry", "colors": ["Red", "Black  
      "] } ],  
  "recipe": { "materials": [  
    { "item": "softwood", "count": 8 },  
    { "item": "iron nugget", "count": 3 } ] } }
```

Embed when the data

- is **always read with** the parent (variations with the item)
- is **small and bounded** (two variations, six materials, never a million)
- **belongs to one parent** (this recipe makes this guitar)
- changes **together** with the parent

In the CSVs: two rows in `housewares.csv`, one row with six numbered material columns in `recipes.csv`. Here: one document, and `{"recipe.materials.item": "softwood"}` finds it.

Referencing

```
// villagers
{ "_id": 1, "name": "Admiral",
  "furniture": [717, 1849, 7047, 2736] }

// items
{ "_id": 717, "name": "clay furnace", "sell": 82
  5 }
{ "_id": 1849, "name": "kotatsu", "sell": 92
  5 }

// what Admiral owns: a second query
db.items.find({ _id: { $in: [717, 1849, 7047] } }
)
```

Reference when the data

- is **shared**: ten villagers own the clay furnace, and its price changes in one place
- is **read on its own**: the item page does not care who owns it
- is **large or unbounded**: a villager's visit log grows forever
- has its **own lifecycle**: items exist without any villager

The reference is a plain value (**manual reference**); MongoDB does not check it. Nothing prevents **furniture: [9999]**: no foreign key, no cascade (*CRUD and Queries*).

Many-to-Many: Villagers and Items

Ids on one side (the usual answer)

```
{ "name": "Admiral",  
  "furniture": [717, 1849] }  
{ "name": "Boone",  
  "furniture": [717, 4029] }  
// who owns the clay furnace?  
db.villagers.find({ furniture: 717 })
```

Ids on both sides (when both directions are hot)

```
{ "_id": 717, "name": "clay furnace",  
  "ownedBy": [1, 57, 88, ...] }  
// two places to keep in sync  
// on every change
```

- The SQL junction table (**villager_furniture**) becomes an **array of ids**; MongoDB queries and indexes inside arrays (*Indexes and Schema Validation*)
- Put the array on the side that is **read with** it and **bounded**: a villager owns a dozen items, a popular item is owned by hundreds
- A separate junction collection is possible and rarely worth it

Embed or Reference?

Show of hands, one relationship at a time: embed, or reference?

1. An item and its two or three colour variations
2. A villager and the villagers who have visited its house (a log, one entry per visit, forever)
3. A recipe and its up to six materials
4. A villager and its favourite song, which 30 other villagers also have as their favourite
5. A player and the settings of their app (theme, language, notifications)
6. An item and the shop that sells it, where the shop has an address, opening hours and a manager

Embed or Reference? Answers

1. **Embed**: small, bounded, always shown with the item, belongs to it
2. **Reference** (own collection **visits** with **villagerId**): unbounded growth would blow the document; the log is read on its own
3. **Embed**: at most six, meaningless without the recipe
4. **It depends**: embed the song *title* (small, rarely changes; 30 copies are fine) or reference by id if the song document is rich and edited. Duplicating a name is normal here.
5. **Embed**: one owner, read together, tiny
6. **Reference**: the shop is shared by many items, read on its own, changes independently

The tell

Ask “if I change this, how many documents change?” and “does anyone read this without its parent?”

The Four Questions

Question	Embed	Reference
Is it read with the parent?	always	often on its own
How many?	a few, bounded	many, or growing without bound
Who else uses it?	only this parent	shared across parents
How often does it change?	with the parent	on its own schedule

- One hard limit behind all four: a document is at most **16 MB**. Unbounded arrays hit it eventually.
- Neither answer is final: a reference can be **extended** by embedding the two fields you always need (the item's name next to its id), a copy that is cheap to keep
- Model for the **queries you will run**, not for the entities you can name. That is the opposite of ER modelling.

Why Embedding Reads Faster

Embedded

- one document, one disk read, one network round trip
- related data sits together on disk (locality)
- one atomic write updates parent and children

Referenced

- two queries, or one **\$lookup** the server executes as two
- each round trip costs more than the query itself
- smaller documents: more of them fit in memory and in an index page

The trap

Embedding an array that grows with use (visits, messages, log lines) makes every read of the parent slower and every update rewrite more bytes, until the 16 MB wall. Growth is the signal to reference.

The Course Dataset, Modelled

```
// items: one polymorphic collection, variations and recipe embedded
{ "_id": 383, "name": "acoustic guitar", "type": "houseware", "sell": 3210,
  "variations": [ { "name": "Natural", "colors": ["Beige", "Brown"] }, ... ],
  "recipe": { "materials": [ { "item": "softwood", "count": 8 }, ... ] } }

// villagers: the home embedded, the furniture referenced by item _id
{ "_id": 1, "name": "Admiral", "species": "Bird", "personality": "Cranky",
  "styles": ["Cool"], "home": { "wallpaper": "dirt-clod wall", "flooring": "tatami" }
,
  "furniture": [717, 1849, 7047, 2736, 787, 5970, 3449, 3622, 3802, 4106] }
```

- 30 CSV files became **two collections**; `recipes.csv` disappeared into the items it describes
- Every decision above answers the four questions; the hands-on asks you to make them yourself, on paper first

Spot the Problems

With your neighbour. Four documents from one **villagers** collection; three have one thing wrong. Find them.

```
{ "name": "Admiral", "birthday": { "month": 1, "day": 27 }, "furniture": [717, 1849]
  }
{ "Name": "Agent S", "birthday": { "month": 7, "day": 2 }, "furniture": [717] }
{ "name": "Ankha", "birthday": "22-Sep", "furniture": [3802] }
{ "name": "Apollo", "birthday": { "month": 7, "day": 4 }, "furniture": "5970;3449"
  }
```

Spot the Problems: Answers

1. Nothing wrong: the reference document.
2. **Field name:** `Name` is a different field from `name`. `find({name: "Agent S"})` returns nothing, and no error.
3. **Type:** a string where the others hold `{month, day}`. `find({"birthday.month": 9})` does not find Ankha.
4. **Structure:** the CSV's semicolon string instead of an array. `find({furniture: 5970})` does not find Apollo.

What SQL would have done

Refused all three at **INSERT**. MongoDB accepts them silently, and the bug appears weeks later in a query that returns too little. *Indexes and Schema Validation* gives you the refusal back, when you ask for it.

Living Without an Enforced Schema

- **Name your fields once**, write it down, and use the same spelling everywhere (`camelCase` in this course)
- **Validate at the edge**: the code that writes a document checks it first; later, a collection validator does it in the database
- **Carry a version**: `schemaVersion` in every document, and a migration for every change
- **Convert on import**: the CSV's `"5970;3449"`, `"NA"` and `"NFS"` become arrays, missing fields and `null` before they are stored, not after
- **Audit**: a pipeline that counts documents per field name and per type finds the drift (*Aggregation I*)

Freedom with a bill attached

A flexible schema moves the discipline from the database to the team. It does not remove it.

Quick Retrieval: Test Yourself

Before the summary: can you answer these from memory?

1. What are the four questions that decide between embedding and referencing?
2. A villager's furniture is stored as an array of item ids. What SQL structure did that replace, and how do you find every owner of item 717?
3. Why is a growing array inside a document a problem, and what is the hard limit?
4. What does a type field do in a polymorphic collection?
5. Name the three ways a document can silently go wrong without a validator.

Think first, then we go through the answers.

Quick Retrieval: Answers

1. Read with the parent? How many, and bounded? Shared with other parents? Changes with the parent or on its own?
2. The junction table `villager_furniture`; `db.villagers.find({furniture: 717})`.
3. Every read and update of the parent carries the whole array; a document may not exceed 16 MB.
4. It names the shape, so code and queries can select one shape (`{type: "fish"}`) among several in one collection.
5. A misspelt field name, a wrong type, a different structure (a string where an array belongs).

Summary: Document Modelling

Documents

- Collections hold documents; documents hold fields, arrays and other documents
- **_id** is the primary key; the ObjectId is unique without coordination
- One collection may hold several shapes; a type field tells them apart

Evolution

- Versions coexist; a **schemaVersion** field and lazy migration replace **ALTER TABLE**

Embed or reference

- Embed first: read together, bounded, owned, changes together
- Reference for shared, independent, unbounded data
- Many-to-many: an array of ids on the bounded side
- Model for the queries, not for the entities

The bill

- No enforced schema: naming, edge validation, versioning are yours

Details: One Collection or Many?

One collection

- Documents with similar shape and the same access pattern (all catalogue items)
- Shared indexes; one query across shapes; atomic updates within a document

Decide by query pattern and indexing need, not by “it is a different entity”. Thirty catalogue CSVs are one collection; villagers are another.

Several collections

- Different shape *and* different access (villagers, orders, sessions)
- Own indexes, own growth, own lifetime

Details: Ids Inside Embedded Documents

- **No id**: the usual case (an address, a variation). The element is addressed by position or by a field value.
- **A small integer or a name**: when the application needs to point at one element ("variation 2", "**Cherry**")
- **An ObjectId**: only when the embedded element is also referenced from somewhere else. That is a hint it might belong in its own collection.

Details: DBRef

```
{ "name": "Admiral",  
  "favoriteSong": { "$ref": "songs", "$id": ObjectId("66f1..."), "$db": "catalogue" }  
}
```

- A convention that names the collection (and database) next to the id, so a generic tool can follow it
- Drivers do not resolve it for you; it costs space and gains nothing when you know the target collection anyway
- MongoDB's own guidance: use manual references unless you need references into *different* collections from one field

Details: What Survives from Normal Forms

- **1NF's spirit survives:** no "5970;3449" strings, no **Style 1/Style 2**. The document answer is an array, not a second table.
- **2NF and 3NF are about one thing:** keep together what depends on the same key. In a document that means: embed what belongs to this parent, reference what has its own key.
- **Deliberate denormalisation:** copying the item's name next to its id in **furniture** is a 3NF violation, done on purpose because the name is read on every villager page and changes never. Know that you are doing it, and know where the copies are.

Details: Large Documents

- The 16 MB limit is a BSON limit; long before it, a document of a few hundred kilobytes is already slow to read, ship and rewrite
- **Files** (images, PDFs) go to GridFS or, better, to object storage with the URL in the document
- **Growing arrays** move to their own collection, or are **bucketed**: one document per villager and month of visits
- **Archival**: old entries move to a cold collection; the hot document stays small

Details: MongoDB Atlas

- The hosted MongoDB: clusters in any cloud region, backups, monitoring, a free tier
- Adds **Atlas Search** (a Lucene index next to the data, see *Search I*) and **Charts** (dashboards on collections)
- This course runs everything locally in Docker; the driver code is identical, only the connection string changes